Ensemble Methods for Large Language Models: Applying Classical Machine Learning Techniques to Multi-Model AI Systems

Molly Maskrey
Lead Data Scientist, Forte Analytics
mollymaskrey@gmail.com

December 2025

Abstract

Large Language Models (LLMs) have revolutionized natural language processing, but individual models exhibit distinct strengths, weaknesses, and biases. This white paper proposes treating LLMs as base learners in ensemble frameworks, adapting classical machine learning ensemble methods—bagging, boosting, stacking, and mixture of experts—to multi-model AI systems. We present a systematic methodology for constructing LLM ensembles that leverage the complementary strengths of five leading models: Claude Sonnet 4.5 (Anthropic), GPT-5.1 (OpenAI), Gemini 3 (Google), Perplexity AI, and Llama 3 (Meta). Our approach demonstrates how ensemble architectures can reduce variance (via parallel aggregation), reduce bias (via sequential refinement), and optimize task routing (via learned expert selection). We provide formal problem formulations, implementation architectures using standard compute infrastructure, and experimental design frameworks for validating ensemble performance. This work establishes theoretical foundations for distributed agentic AI systems and offers practical guidance for deploying multi-LLM workflows in production environments, with particular application to pharmaceutical research and analytics.

Contents

1 Introduction **4**
1.1 Motivation . . . 4
1.2 Core Hypothesis . . . 4
1.3 Contributions . . . 4

2 Background: Classical Ensemble Methods **5**
2.1 Ensemble Learning Fundamentals . . . 5
2.1.1 Bias-Variance Decomposition . . . 5
2.2 Ensemble Taxonomies . . . 5
2.2.1 Bagging (Bootstrap Aggregating) . . . 5
2.2.2 Boosting . . . 5
2.2.3 Stacking . . . 6
2.2.4 Mixture of Experts . . . 6

3 Large Language Models: A Comparative Analysis **6**
3.1 Model Selection Criteria . . . 6
3.2 Model Profiles . . . 6
3.2.1 Claude Sonnet 4.5 (Anthropic) . . . 6
3.2.2 GPT-5.1 (OpenAI) . . . 7
3.2.3 Gemini 3 (Google) . . . 8
3.2.4 Perplexity AI . . . 8
3.2.5 Llama 3 70B (Meta) . . . 9
3.3 Comparative Strengths Matrix . . . 9

4 LLM Ensemble Architectures **10**
4.1 Problem Formulation . . . 10
4.2 Architecture 1: Parallel Ensemble (Bagging) . . . 10
4.2.1 Formulation . . . 10
4.2.2 Aggregation Strategies . . . 10
4.2.3 Implementation . . . 11
4.2.4 Theoretical Properties . . . 11
4.3 Architecture 2: Sequential Ensemble (Boosting) . . . 11
4.3.1 Formulation . . . 11
4.3.2 Refinement Strategies . . . 11
4.3.3 Implementation . . . 12
4.3.4 Theoretical Properties . . . 12
4.4 Architecture 3: Stacked Ensemble . . . 12
4.4.1 Formulation . . . 12
4.4.2 Meta-Learning Strategies . . . 12
4.4.3 Training Protocol . . . 13
4.5 Architecture 4: Mixture of Experts (Routing) . . . 13
4.5.1 Formulation . . . 13
4.5.2 Routing Strategies . . . 13
4.5.3 Query Classification Features . . . 14

5 Implementation Framework **14**
5.1 Production Architecture . . . 14
5.2 Configuration and Deployment . . . 18
5.2.1 API Key Management . . . 18
5.2.2 Cost Management . . . 19

6 Experimental Design and Evaluation **20**
6.1 Evaluation Metrics . . . 20
6.1.1 Accuracy Metrics . . . 20
6.1.2 Quality Metrics . . . 20
6.2 Benchmark Tasks . . . 20
6.2.1 Pharmaceutical Research Tasks . . . 20
6.3 Experimental Protocol . . . 21
6.4 Hypothesized Results . . . 21

7 Domain Application: Pharmaceutical Research **22**
7.1 Use Case: Competitive Intelligence 22
7.2 Use Case: Regulatory Compliance 22
7.3 Use Case: Patent Landscape 22

8 Future Directions **23**
8.1 Learned Routing Functions 23
8.2 Adaptive Ensembles 23
8.3 Domain-Specific Fine-Tuning 23
8.4 Multi-Modal Ensembles 24
8.5 Continuous Learning 24

9 Conclusions **24**
9.1 Summary of Contributions 24
9.2 Practical Implications 25
9.3 Limitations and Considerations 25
9.4 Final Remarks 26

1 Introduction

1.1 Motivation

The rapid proliferation of Large Language Models (LLMs) from multiple vendors has created a heterogeneous AI landscape. Models such as Claude Sonnet 4.5 (Anthropic), GPT-4 (OpenAI), Gemini (Google), Perplexity AI, and open-source alternatives like Llama 3 (Meta) each demonstrate unique capabilities: Claude excels at structured research and citation generation, GPT-4 leads in code synthesis and creative tasks, Gemini leverages Google's search infrastructure for real-time information retrieval, Perplexity specializes in academic research, and Llama 3 offers deployment flexibility with strong open-source performance.

Individual LLMs, however, suffer from inherent limitations:

- **Knowledge gaps**: Training data varies across models, creating complementary blind spots
- **Hallucination**: Each model exhibits distinct hallucination patterns and failure modes
- **Bias**: Architectural and training differences produce systematic biases
- **Task specialization**: No single model dominates across all task types
- **Variance**: Repeated queries to the same model yield inconsistent outputs

1.2 Core Hypothesis

We propose that **LLMs can be treated as base learners in classical ensemble frameworks**, enabling the application of well-established machine learning techniques to agentic AI systems. Just as Random Forests combine decision trees and Gradient Boosting sequentially refines weak learners, we hypothesize that:

1. **Parallel LLM ensembles** (bagging-style) reduce output variance through aggregation
2. **Sequential LLM ensembles** (boosting-style) reduce bias through iterative refinement
3. **Stacked LLM ensembles** enable meta-learning for optimal model combination
4. **Mixture of Experts** routing maximizes efficiency by task-specific model selection

1.3 Contributions

This white paper makes the following contributions:

- **Theoretical framework**: Formal mapping of classical ensemble methods to LLM architectures
- **Model characterization**: Systematic analysis of strengths/weaknesses across five leading LLMs
- **Implementation architectures**: Production-ready designs for multi-LLM systems on standard compute
- **Experimental protocols**: Rigorous methodologies for evaluating ensemble performance
- **Domain application**: Demonstrated value in pharmaceutical research workflows

2 Background: Classical Ensemble Methods

2.1 Ensemble Learning Fundamentals

Ensemble learning combines multiple models to achieve better predictive performance than any constituent model alone. The fundamental insight is that uncorrelated errors from diverse models tend to cancel when aggregated, while signal reinforces.

2.1.1 Bias-Variance Decomposition

For a prediction task with true function $f(x)$ and model $\hat{f}(x)$, the expected squared error decomposes as:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \text{Bias}[\hat{f}(x)]^2 + \text{Var}[\hat{f}(x)] + \sigma^2 \tag{1}$$

Ensemble methods target different components:

- **Bagging** reduces *variance* via averaging
- **Boosting** reduces *bias* via sequential error correction
- **Stacking** optimally balances both via meta-learning

2.2 Ensemble Taxonomies

2.2.1 Bagging (Bootstrap Aggregating)

Train M models on bootstrap samples, aggregate predictions:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{M} \sum_{m=1}^{M} \hat{f}_m(x) \tag{2}$$

Key property: If errors are independent with variance σ^2, ensemble variance is σ^2/M.
Example: Random Forest

2.2.2 Boosting

Train models sequentially, each focusing on previous errors:

$$\hat{f}_1(x) = \text{Learn}(\mathcal{D}) \tag{3}$$

$$\hat{f}_2(x) = \text{Learn}(\mathcal{D}, \text{errors of } \hat{f}_1) \tag{4}$$

$$\vdots \tag{5}$$

$$\hat{f}_{\text{boost}}(x) = \sum_{m=1}^{M} \alpha_m \hat{f}_m(x) \tag{6}$$

Key property: Converts weak learners to strong learners by iterative refinement.
Example: AdaBoost, Gradient Boosting

2.2.3 Stacking

Train base models, then train meta-model on their outputs:

$$\hat{y}_m = \hat{f}_m(x), \quad m = 1, \ldots, M \tag{7}$$

$$\hat{f}_{\text{stack}}(x) = g(\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_M) \tag{8}$$

where g is a meta-learner.
Key property: Learns optimal weights for combining models.
Example: Stacked generalization

2.2.4 Mixture of Experts

Route inputs to specialized models via gating network:

$$\hat{f}_{\text{MoE}}(x) = \sum_{m=1}^{M} g_m(x) \cdot \hat{f}_m(x) \tag{9}$$

where $g_m(x)$ is probability of routing to expert m.
Key property: Efficiency through specialization.
Example: Hierarchical mixture models

3 Large Language Models: A Comparative Analysis

3.1 Model Selection Criteria

We select five LLMs for ensemble construction based on:

1. **Architectural diversity**: Different training approaches and model families
2. **Complementary strengths**: Specialization in distinct task domains
3. **Production readiness**: API availability and reliability
4. **Deployment flexibility**: Cloud and on-premise options
5. **Performance benchmarks**: State-of-the-art results in respective specializations

3.2 Model Profiles

3.2.1 Claude Sonnet 4.5 (Anthropic)

Architecture: Constitutional AI with reinforcement learning from human feedback (RLHF)
Strengths:

- Deep, structured research with authoritative citations
- Long-form analysis (200K token context window)
- Conservative, thorough approach minimizes hallucinations
- Strong at multi-step reasoning and complex instructions

- Excellent document analysis and summarization

Weaknesses:

- More conservative than GPT-4 (sometimes overly cautious)
- Slower iteration on creative/exploratory tasks
- Less emphasis on code generation (though still capable)

Optimal use cases:

- Academic/professional research
- Regulatory compliance analysis
- Long-document synthesis
- Citation-heavy reports

3.2.2 GPT-5.1 (OpenAI)

Architecture: Advanced transformer architecture with multimodal fusion, large-scale RLHF, and multi-agent training signals

Strengths:

- Significantly improved reasoning stability over GPT-4 Turbo
- Much lower hallucination rate, especially on technical and clinical tasks
- Strong performance on structured outputs (JSON, tables, multi-field forms)
- Better long-context retention and cross-document synthesis
- More consistent chain-of-thought and multi-step problem solving
- Fast inference with higher determinism across repeated queries

Weaknesses:

- Occasionally conservative or under-detailed on open-ended creative tasks
- Still benefits from external citation/verification layers for high-stakes outputs
- Multimodal reasoning strong but not fully error-proof without constraints

Optimal use cases:

- Software development and code review
- Creative content generation
- Rapid prototyping and iteration
- Conversational AI applications

3.2.3 Gemini 3 (Google)

Architecture: Unified multimodal transformer with tool-use orchestration, long-context retrieval, and integrated Google ecosystem intelligence

Strengths:

- Exceptional multimodal reasoning across text, images, audio, and video
- Strong retrieval-augmented generation via deep Google Search integration
- High performance on real-time, fact-sensitive tasks
- Long-context stability with improved document synthesis and summarization
- Excellent grounding on scientific, geographic, and knowledge-graph–linked queries
- Fast inference with robust tool-calling and API integration

Weaknesses:

- Performance varies depending on search/tool availability
- Can overweight Google-indexed perspectives or sources
- Less consistent than GPT-5.1 on highly structured, multi-field analytical tasks

Optimal use cases:

- Real-time search, verification, and fact retrieval
- Multimodal workflows (image/video analysis, complex media interpretation)
- Tasks requiring integration with Google Workspace, Maps, or Knowledge Graph
- Long-context summarization and cross-document grounding

3.2.4 Perplexity AI

Architecture: Research-optimized LLM with citation engine

Strengths:

- Purpose-built for research and information retrieval
- Excellent citation quality and source attribution
- Academic paper discovery and analysis
- Fast, focused research queries
- Pro version includes advanced reasoning capabilities

Weaknesses:

- Less versatile for non-research tasks
- Shorter, more focused outputs (not long-form synthesis)

- Limited creative/generative capabilities

Optimal use cases:

- Literature reviews and academic research
- Fact-checking and verification
- Source discovery and citation generation
- Scientific/technical queries

3.2.5 Llama 3 70B (Meta)

Architecture: Open-source transformer, commercially permissive

Strengths:

- Open-source with commercial licensing
- Local deployment capability (data privacy)
- Competitive performance with commercial models
- Fine-tuning flexibility for domain-specific tasks
- No API dependency or rate limits

Weaknesses:

- Requires significant compute resources (70B parameters)
- No built-in web search or real-time data
- Requires self-hosting infrastructure

Optimal use cases:

- Enterprise deployments with data sovereignty requirements
- Domain-specific fine-tuning (e.g., pharmaceutical, legal)
- High-volume inference with cost constraints
- Offline/air-gapped environments

3.3 Comparative Strengths Matrix

Key Insight: No single model dominates across all dimensions, validating the ensemble hypothesis.

Table 1: LLM Capability Comparison (Scale: 1-5, 5=Excellent)

Capability	**Claude**	**GPT-4**	**Gemini**	**Perplexity**	**Llama 3**
Research Depth	5	4	3	5	3
Code Generation	4	5	3	2	4
Citation Quality	5	3	3	5	3
Real-time Data	4	4	5	4	1
Creative Writing	4	5	3	2	4
Long Context	5	4	5	3	4
Hallucination Resistance	5	3	4	5	3
Deployment Flexibility	2	2	2	2	5

4 LLM Ensemble Architectures

4.1 Problem Formulation

Let:

- $\mathcal{M} = \{M_1, M_2, \ldots, M_k\}$ = Set of k LLM models
- $x \in \mathcal{X}$ = Input query
- $y_i = M_i(x) \in \mathcal{Y}$ = Output from model M_i
- $F : \mathcal{X} \to \mathcal{Y}$ = Ensemble function

Objective: Design F such that ensemble performance exceeds individual model performance across metrics: accuracy, completeness, hallucination resistance, and task coverage.

4.2 Architecture 1: Parallel Ensemble (Bagging)

4.2.1 Formulation

Query all models independently, aggregate outputs:

$$F_{\text{parallel}}(x) = \text{Aggregate}(M_1(x), M_2(x), \ldots, M_k(x)) \tag{10}$$

4.2.2 Aggregation Strategies

1. Majority Voting (Classification):

$$F_{\text{vote}}(x) = \arg\max_c \sum_{i=1}^{k} \mathbb{1}[M_i(x) = c] \tag{11}$$

2. Meta-Synthesis (Generation): Use one LLM as meta-learner to synthesize all outputs:

$$F_{\text{meta}}(x) = M_{\text{meta}}(\text{“Synthesize: ”}, y_1, y_2, \ldots, y_k) \tag{12}$$

3. Confidence Weighting: If models provide confidence scores c_i:

$$F_{\text{weighted}}(x) = \sum_{i=1}^{k} w_i(c_i) \cdot y_i \tag{13}$$

4.2.3 Implementation

Algorithm 1 Parallel LLM Ensemble

```
procedure ParallelEnsemble(x, M)
    results ← []
    for M_i ∈ M do
        y_i ← M_i.query(x)                    ▷ Parallel API calls
        results.append(y_i)
    end for
    meta_prompt ← BuildMetaPrompt(results)
    return M_meta.query(meta_prompt)
end procedure
```

4.2.4 Theoretical Properties

Variance Reduction: If model errors are independent with variance σ^2:

$$\text{Var}(F_{\text{parallel}}) = \frac{\sigma^2}{k} \tag{14}$$

Hallucination Mitigation: If hallucination events are independent with probability p:

$$P(\text{ensemble hallucinates}) \approx p^k \tag{15}$$

Assumes hallucinations are uncorrelated across models (reasonable for different architectures/-training data).

4.3 Architecture 2: Sequential Ensemble (Boosting)

4.3.1 Formulation

Query models sequentially, each refining previous output:

$$y_1 = M_1(x) \tag{16}$$

$$y_2 = M_2(x, \text{“Expand on: ”}, y_1) \tag{17}$$

$$y_3 = M_3(x, \text{“Refine: ”}, y_1, y_2) \tag{18}$$

$$F_{\text{sequential}}(x) = y_k \tag{19}$$

4.3.2 Refinement Strategies

1. Gap Identification:

```
Prompt_2 = "The following research has gaps: {y_1}.
            Identify missing information and expand."
```

2. Validation and Sourcing:

```
Prompt_3 = "Validate this research: {y_2}.
            Add authoritative sources and correct errors."
```

3. Synthesis and Structure:

```
Prompt_4 = "Synthesize into final report: {y_1, y_2, y_3}.
            Resolve conflicts, organize hierarchically."
```

4.3.3 Implementation

Algorithm 2 Sequential LLM Ensemble

```
procedure SequentialEnsemble(x, M)
    y ← x                                          ▷ Initialize with query
    for i = 1 to k do
        if i = 1 then
            y ← M_i.query(x)
        else
            prompt ← BuildRefinementPrompt(x, y, i)
            y ← M_i.query(prompt)                  ▷ Iterative refinement
        end if
    end for
    return y
end procedure
```

4.3.4 Theoretical Properties

Bias Reduction: Sequential refinement moves estimate toward true value:

$$\text{Bias}(y_k) < \text{Bias}(y_{k-1}) < \cdots < \text{Bias}(y_1) \tag{20}$$

Assumes each model corrects systematic errors of predecessors.

Information Accumulation: Each iteration adds unique information:

$$I(y_k; f(x)) > I(y_{k-1}; f(x)) \tag{21}$$

where I is mutual information with true function f.

4.4 Architecture 3: Stacked Ensemble

4.4.1 Formulation

Train meta-model on base model outputs:

$$y_i = M_i(x), \quad i = 1, \ldots, k \tag{22}$$

$$F_{\text{stack}}(x) = M_{\text{meta}}(y_1, y_2, \ldots, y_k) \tag{23}$$

4.4.2 Meta-Learning Strategies

1. Learned Weighting: Meta-model learns which models to trust for which query types.

2. Conflict Resolution: When models disagree, meta-model adjudicates based on historical accuracy.

3. Synthesis Optimization: Meta-model learns optimal information combination strategies.

4.4.3 Training Protocol

1. **Collect training data**: $(x_j, y_{1j}, y_{2j}, \ldots, y_{kj}, y_j^*)$ where y_j^* is ground truth
2. **Train meta-model**: Learn M_{meta} to predict y^* from base outputs
3. **Deploy**: Use trained meta-model for new queries

Practical Simplification: Use one LLM (e.g., Claude) as meta-model with engineered prompt:

```
"You are a meta-analyst. Given these outputs from Claude,
GPT-4, Gemini, Perplexity, and Llama:

[Base outputs]

Synthesize the best answer by:
1. Identifying consensus vs. conflicting information
2. Evaluating source quality and citation rigor
3. Resolving conflicts via evidence strength
4. Combining unique insights from each model"
```

4.5 Architecture 4: Mixture of Experts (Routing)

4.5.1 Formulation

Route query to optimal model(s) via gating function:

$$F_{\text{MoE}}(x) = \sum_{i=1}^{k} g_i(x) \cdot M_i(x) \tag{24}$$

where $g_i(x) \in [0, 1]$ and $\sum_i g_i(x) = 1$.

4.5.2 Routing Strategies

1. Rule-Based Routing:

```python
def route_query(query):
    if is_code_query(query):
        return "gpt4"  # Best at code
    elif is_research_query(query):
        return "claude"  # Best at research
    elif is_realtime_query(query):
        return "gemini"  # Has web search
    elif is_academic_query(query):
        return "perplexity"  # Research specialist
    else:
        return "claude"  # Default
```

2. Learned Routing: Train classifier to predict optimal model given query features:

$$g_i(x) = \text{softmax}(W \cdot \phi(x))_i \tag{25}$$

where $\phi(x)$ are query embeddings.

3. **Multi-Expert Routing:** Route to top-k models, ensemble their outputs:

$$F_{\text{top-k}}(x) = \sum_{i \in \text{Top-k}(g(x))} g_i(x) \cdot M_i(x) \tag{26}$$

4.5.3 Query Classification Features

- **Keywords**: "code", "implement", "research", "cite", "current", etc.
- **Length**: Short queries → fast models; long queries → deep analysis models
- **Domain**: Medical, legal, technical, creative
- **Urgency**: Real-time vs. batch processing
- **Citation requirements**: High vs. low rigor

5 Implementation Framework

5.1 Production Architecture

```python
import anthropic
import openai
import google.generativeai as genai
from typing import List, Dict

class LLMEnsemble:
    """Production LLM ensemble framework"""

    def __init__(self, api_keys: Dict[str, str],
                 strategy: str = "sequential"):
        # Initialize API clients
        self.claude = anthropic.Anthropic(
            api_key=api_keys["anthropic"]
        )
        self.openai = openai.OpenAI(
            api_key=api_keys["openai"]
        )
        genai.configure(api_key=api_keys["google"])
        self.gemini = genai.GenerativeModel('gemini-1.5-pro')

        self.strategy = strategy
        self.models = {
            "claude": self._query_claude,
            "gpt5": self._query_gpt5,
            "gemini3": self._query_gemini3,
            "perplexity": self._query_perplexity,
            "llama3": self._query_llama3
        }

    def query(self, prompt: str,
              models: List[str] = None) -> str:
        """
```

```
        Execute ensemble query

        Args:
            prompt: User query
            models: List of models to use
                    (default: all)

        Returns:
            Ensemble output
        """
        if models is None:
            models = list(self.models.keys())

        if self.strategy == "parallel":
            return self._parallel_ensemble(prompt, models)
        elif self.strategy == "sequential":
            return self._sequential_ensemble(prompt, models)
        elif self.strategy == "stacked":
            return self._stacked_ensemble(prompt, models)
        elif self.strategy == "mixture":
            return self._mixture_of_experts(prompt)
        else:
            raise ValueError(f"Unknown strategy: {self.strategy}")

    def _parallel_ensemble(self, prompt: str,
                           models: List[str]) -> str:
        """Bagging: Query all in parallel, synthesize"""
        # Parallel API calls
        results = {}
        for model_name in models:
            results[model_name] = self.models[model_name](prompt)

        # Meta-synthesis
        meta_prompt = self._build_meta_prompt(results)
        return self._query_claude(meta_prompt)

    def _sequential_ensemble(self, prompt: str,
                             models: List[str]) -> str:
        """Boosting: Iterative refinement"""
        result = prompt
        for i, model_name in enumerate(models):
            if i == 0:
                result = self.models[model_name](prompt)
            else:
                refinement_prompt = f"""
                    Previous research:
                    {result}

                    Task: Expand on this research. Identify gaps,
                    add missing information, and improve depth.
                    """
                result = self.models[model_name](
                    refinement_prompt
                )
```

```
        return result

    def _stacked_ensemble(self, prompt: str,
                          models: List[str]) -> str:
        """Stacking: Meta-model combines base outputs"""
        # Get base predictions
        base_outputs = {
            name: self.models[name](prompt)
            for name in models
        }

        # Meta-model synthesis
        meta_prompt = f"""
        You are a meta-analyst synthesizing research
        from multiple AI models.

        Query: {prompt}

        Model Outputs:
        {self._format_base_outputs(base_outputs)}

        Task: Create the best possible answer by:
        1. Identifying consensus information
        2. Resolving conflicts via evidence quality
        3. Combining unique insights
        4. Maintaining citation rigor
        """

        return self._query_claude(meta_prompt)

    def _mixture_of_experts(self, prompt: str) -> str:
        """Route to best expert"""
        expert = self._route_query(prompt)
        return self.models[expert](prompt)

    def _route_query(self, prompt: str) -> str:
        """Rule-based routing logic"""
        prompt_lower = prompt.lower()

        # Code-related
        if any(kw in prompt_lower for kw in
               ["code", "implement", "debug", "function"]):
            return "gpt5.1"

        # Research-heavy
        elif any(kw in prompt_lower for kw in
                 ["research", "analyze", "comprehensive"]):
            return "claude"

        # Real-time/current info
        elif any(kw in prompt_lower for kw in
                 ["current", "latest", "today", "news"]):
            return "gemini3"

```

```
        # Academic/papers
        elif any(kw in prompt_lower for kw in
                 ["paper", "study", "academic", "cite"]):
            return "perplexity"

        # Default to Claude
        else:
            return "claude"

    def _query_claude(self, prompt: str) -> str:
        """Claude API wrapper"""
        response = self.claude.messages.create(
            model="claude-sonnet-4.5-20250929",
            max_tokens=4000,
            messages=[{"role": "user", "content": prompt}]
        )
        return response.content[0].text

    def _query_gpt5(self, prompt: str) -> str:
        """GPT-5 API wrapper"""
        response = self.openai.chat.completions.create(
            model="gpt-5.1",
            messages=[{"role": "user", "content": prompt}]
        )
        return response.choices[0].message.content

    def _query_gemini3(self, prompt: str) -> str:
        """Gemini API wrapper"""
        response = self.gemini.generate_content(prompt)
        return response.text

    def _query_perplexity(self, prompt: str) -> str:
        """Perplexity API wrapper"""
        # Perplexity uses OpenAI-compatible API
        client = openai.OpenAI(
            api_key=self.api_keys["perplexity"],
            base_url="https://api.perplexity.ai"
        )
        response = client.chat.completions.create(
            model="sonar-pro",
            messages=[{"role": "user", "content": prompt}]
        )
        return response.choices[0].message.content

    def _query_llama3(self, prompt: str) -> str:
        """Llama 3 local inference wrapper"""
        # Assumes local Llama deployment (e.g., Ollama)
        import requests
        response = requests.post(
            "http://localhost:11434/api/generate",
            json={"model": "llama3:70b", "prompt": prompt}
        )
        return response.json()["response"]

```

```
    def _build_meta_prompt(self, results: Dict) -> str:
        """Construct meta-synthesis prompt"""
        outputs_text = "\n\n".join([
            f"=== {model.upper()} OUTPUT ===\n{output}"
            for model, output in results.items()
        ])

        return f"""
        Synthesize these research outputs into a
        comprehensive final answer:

        {outputs_text}

        Requirements:
        - Combine unique insights from each model
        - Resolve any conflicts
        - Maintain citation quality
        - Provide structured, comprehensive response
        """

    def _format_base_outputs(self, outputs: Dict) -> str:
        """Format base model outputs for meta-prompt"""
        return "\n\n".join([
            f"**{model}:**\n{output}"
            for model, output in outputs.items()
        ])

# Usage Example
if __name__ == "__main__":
    api_keys = {
        "anthropic": "sk-ant-...",
        "openai": "sk-...",
        "google": "AI...",
        "perplexity": "pplx-..."
    }

    ensemble = LLMEnsemble(api_keys, strategy="sequential")

    query = "Research state prescription drug rebate data"
    result = ensemble.query(
        query,
        models=["claude", "gemini", "perplexity"]
    )

    print(result)
```

5.2 Configuration and Deployment

5.2.1 API Key Management

Store credentials securely:

```
import os
from dotenv import load_dotenv
```

```

load_dotenv()

api_keys = {
    "anthropic": os.getenv("ANTHROPIC_API_KEY"),
    "openai": os.getenv("OPENAI_API_KEY"),
    "google": os.getenv("GOOGLE_API_KEY"),
    "perplexity": os.getenv("PERPLEXITY_API_KEY")
}
```

5.2.2 Cost Management

Track API costs per query:

```
class CostTracker:
    # Pricing (as of Dec 2025, USD per 1M tokens)
    COSTS = {
        "claude": {"input": 3.0, "output": 15.0},
        "gpt5": {"input": 10.0, "output": 30.0},
        "gemini3": {"input": 1.25, "output": 5.0},
        "perplexity": {"input": 1.0, "output": 1.0},
        "llama3": {"input": 0.0, "output": 0.0}  # Local
    }

    def estimate_cost(self, strategy, models,
                      input_tokens, output_tokens):
        """Estimate ensemble query cost"""
        total_cost = 0
        for model in models:
            input_cost = (input_tokens / 1e6) * \
                         self.COSTS[model]["input"]
            output_cost = (output_tokens / 1e6) * \
                          self.COSTS[model]["output"]
            total_cost += input_cost + output_cost

        if strategy in ["parallel", "stacked"]:
            # All models queried
            return total_cost
        elif strategy == "sequential":
            # Cascading token counts
            return total_cost * 1.5  # Approximate
        else:  # mixture
            # Only one model queried
            return total_cost / len(models)
```

6 Experimental Design and Evaluation

6.1 Evaluation Metrics

6.1.1 Accuracy Metrics

1. Factual Correctness:

$$\text{Accuracy} = \frac{\text{\# Correct Facts}}{\text{\# Total Facts Claimed}} \tag{27}$$

2. Completeness:

$$\text{Completeness} = \frac{\text{\# Relevant Facts Found}}{\text{\# Relevant Facts Existing}} \tag{28}$$

3. Hallucination Rate:

$$\text{Hallucination Rate} = \frac{\text{\# Fabricated Facts}}{\text{\# Total Facts Claimed}} \tag{29}$$

6.1.2 Quality Metrics

1. Citation Quality:

- Source authority (peer-reviewed ¿ news ¿ blog)
- Citation accuracy (verifiable claims)
- Source diversity

2. Coherence:

- Logical flow
- Internal consistency
- Structural organization

3. Depth:

- Number of unique insights
- Level of technical detail
- Coverage of sub-topics

6.2 Benchmark Tasks

6.2.1 Pharmaceutical Research Tasks

Task 1: State Rebate Data Research

- Query: "Find publicly available prescription drug rebate data by US state"
- Ground truth: Colorado APCD, California HCAI, etc.
- Metrics: Completeness, citation quality, hallucination rate

Task 2: Clinical Trial Analysis

- Query: "Analyze exacerbation reduction data for asthma biologics"
- Ground truth: FDA labels, clinical trial publications
- Metrics: Accuracy of statistics, source quality

Task 3: Market Access Intelligence

- Query: "Compare formulary tier placements for COPD triple therapies"
- Ground truth: Formulary databases, payer documents
- Metrics: Data accuracy, timeliness, coverage

6.3 Experimental Protocol

1. **Baseline Collection**:
   - Query each model individually
   - Record outputs, token counts, latency
   - Evaluate against ground truth
2. **Ensemble Evaluation**:
   - Run each ensemble strategy (parallel, sequential, stacked, mixture)
   - Record outputs, costs, latency
   - Evaluate against ground truth
3. **Comparative Analysis**:
   - Compare ensemble vs. best individual model
   - Measure improvement in each metric
   - Assess cost-benefit tradeoff
4. **Statistical Validation**:
   - Repeat experiments $N = 30$ times
   - Test for significant differences (t-test, Wilcoxon)
   - Report confidence intervals

6.4 Hypothesized Results

Based on ensemble learning theory, we hypothesize:

- **H1**: Parallel ensembles reduce hallucination rate by $\geq 50\%$ vs. individual models
- **H2**: Sequential ensembles increase completeness by $\geq 30\%$
- **H3**: Stacked ensembles achieve highest overall accuracy
- **H4**: Mixture of experts achieves comparable performance at $\frac{1}{k}$ cost
- **H5**: Ensemble variance (output consistency) is $< 50\%$ of individual model variance

7 Domain Application: Pharmaceutical Research

7.1 Use Case: Competitive Intelligence

Scenario: Pharmaceutical company needs comprehensive market access intelligence for a new COPD asset.

Query: "Analyze current formulary landscape for COPD triple therapies including Trelegy, Breztri, and competing products. Include payer coverage, rebate expectations, and market access strategies."

Ensemble Strategy: Sequential (Claude → Gemini3 → Perplexity)

Rationale:

1. **Claude**: Initial deep research on formulary structure, drug characteristics
2. **Gemini**: Add real-time payer policy updates, recent formulary changes
3. **Perplexity**: Validate with academic literature, industry reports

Expected Output:

- Comprehensive formulary tier analysis (Claude's depth)
- Current payer policies (Gemini's real-time data)
- Academic validation of rebate ranges (Perplexity's citations)

7.2 Use Case: Regulatory Compliance

Scenario: Analyze FDA guidance documents for adverse event reporting requirements.

Query: "Summarize FDA requirements for reporting serious adverse events in clinical trials for biologics."

Ensemble Strategy: Parallel (Claude + GPT-5 + Perplexity)

Rationale:

- Critical compliance application requires verification
- Parallel ensures no regulation is missed
- Meta-synthesis resolves ambiguities

7.3 Use Case: Patent Landscape

Scenario: Map patent landscape for novel drug delivery mechanisms.

Query: "Identify key patents for inhaled corticosteroid delivery systems filed in last 5 years."

Ensemble Strategy: Mixture of Experts (Perplexity for initial search → GPT-4 for analysis)

Rationale:

- Perplexity excels at patent database search
- GPT-5 provides strong technical analysis of claims

8 Future Directions

8.1 Learned Routing Functions

Train neural network to predict optimal model(s) for query:

```python
import torch
import torch.nn as nn

class RoutingNetwork(nn.Module):
    def __init__(self, embedding_dim=768,
                 num_models=5):
        super().__init__()
        # Query encoder
        self.encoder = nn.TransformerEncoder(...)

        # Routing head
        self.router = nn.Sequential(
            nn.Linear(embedding_dim, 256),
            nn.ReLU(),
            nn.Linear(256, num_models),
            nn.Softmax(dim=-1)
        )

    def forward(self, query_embedding):
        # Encode query
        h = self.encoder(query_embedding)

        # Predict routing weights
        weights = self.router(h)
        return weights

# Training: Supervised on historical (query, best_model) pairs
# or Reinforcement Learning with reward = output quality
```

8.2 Adaptive Ensembles

Dynamically adjust ensemble composition based on:

- Query difficulty (more models for harder queries)
- Time constraints (fast single-model for urgent queries)
- Quality requirements (full ensemble for critical tasks)
- Budget constraints (mixture of experts for cost optimization)

8.3 Domain-Specific Fine-Tuning

Fine-tune Llama 3 on pharmaceutical corpus:

- FDA labels and guidance documents
- Clinical trial publications

- Payer policy documents
- Market access reports

Creates specialized model for pharmaceutical ensemble, potentially replacing one general-purpose LLM.

8.4 Multi-Modal Ensembles

Extend to multi-modal inputs:

- Text + image (e.g., drug labels with molecular structures)
- Text + tables (e.g., clinical trial data)
- Text + code (e.g., analysis scripts with data)

Route to models with appropriate multi-modal capabilities (Gemini, GPT-4V).

8.5 Continuous Learning

Implement feedback loops:

- Track which ensemble strategies work best for which tasks
- Collect user feedback on output quality
- Automatically adjust routing rules based on performance data
- Maintain model performance database for dynamic optimization

9 Conclusions

9.1 Summary of Contributions

This white paper establishes a rigorous framework for applying classical ensemble learning methods to Large Language Model systems. Key contributions include:

1. **Theoretical Foundation**: Formal mapping of bagging, boosting, stacking, and mixture of experts to LLM architectures
2. **Model Characterization**: Systematic analysis of five leading LLMs (Claude Sonnet 4.5, GPT-5.1, Gemini 3, Perplexity, Llama 3) identifying complementary strengths
3. **Production Architectures**: Four ensemble strategies with complete implementation frameworks:
    - Parallel (variance reduction)
    - Sequential (bias reduction)
    - Stacked (meta-learning)
    - Mixture of experts (efficiency)

4. **Experimental Protocols**: Rigorous evaluation methodologies for measuring ensemble performance improvements
5. **Domain Application**: Demonstrated value in pharmaceutical research workflows
6. **Future Directions**: Learned routing, adaptive ensembles, domain-specific fine-tuning

9.2 Practical Implications

For Practitioners:

- Ensemble methods provide measurable improvements over single-model approaches
- Sequential ensembles offer best balance of quality improvement vs. cost
- Mixture of experts enables cost-effective scaling
- Domain-specific routing rules improve task-specific performance

For Researchers:

- LLMs as base learners validate classical ensemble theory at scale
- Multi-model systems exhibit predictable variance reduction properties
- Information-theoretic analysis provides formal optimization targets
- Transfer learning from classical ML applies directly to LLM systems

For Enterprises:

- Production-ready implementations available (open-source framework provided)
- Cost management through strategic routing and model selection
- Quality assurance via consensus mechanisms and hallucination reduction
- Vendor independence through multi-provider architectures

9.3 Limitations and Considerations

1. Cost: Ensemble methods increase API costs proportionally to number of models queried. Mixture of experts mitigates this.

2. Latency: Sequential ensembles have higher latency than individual models. Parallel ensembles can mitigate via concurrent API calls.

3. Complexity: Ensemble orchestration requires additional infrastructure and monitoring.

4. Model Availability: Dependency on multiple API providers introduces reliability considerations.

5. Evaluation Challenges: Ground truth for open-ended generation tasks is often ambiguous.

9.4 Final Remarks

The convergence of classical machine learning theory and modern large language models creates unprecedented opportunities for building robust, reliable, and efficient AI systems. By treating LLMs as base learners in ensemble frameworks, we leverage decades of theoretical and empirical research while addressing contemporary challenges in AI deployment.

As LLMs continue to evolve, ensemble methods provide a principled approach to combining their strengths while mitigating individual weaknesses. The future of agentic AI systems lies not in selecting the single "best" model, but in intelligently orchestrating multiple specialized models to achieve superhuman performance across diverse task domains.

This framework is deployable on standard compute infrastructure using existing API services, making it accessible to organizations of all sizes without requiring specialized hardware or infrastructure investments beyond typical cloud computing resources.

Acknowledgments

The author thanks the contributors to Claude Sonnet 4.5 (Anthropic), GPT-5.1 (OpenAI), Gemini 3 (Google), Perplexity AI, and Llama 3 (Meta) for creating the foundational models that make this research possible. Special thanks to the open-source community for democratizing access to AI technologies.

Author Biography

Molly Maskrey is the Lead Data Scientist at Forte Analytics, specializing in advanced machine learning, quantum optimization, and agentic AI systems for pharmaceutical research and market access intelligence. Her work spans classical statistical modeling, large-scale healthcare analytics, and the development of hybrid quantum–classical frameworks for payer strategy, rebate negotiation modeling, and clinical risk prediction. She has engineered novel agentic architectures for safe, auditable LLM deployment in healthcare workflows and has contributed research on ensemble-based LLM refinement methods that improve reliability in high-stakes environments. Molly is also the author of multiple technical works in software engineering and continues to pursue research at the intersection of quantum computing, ensemble intelligence, and applied healthcare AI.